

Short summary of the math / logic in the requested sections

- **Problem statement & notation:** conditional generation with input x and output token sequence y . Concatenate $z = [x; y]$. Activations at timestep i are $h_i \in \mathbb{R}^d$ (concatenation of layer activations). The autoregressive recurrence is written as

$$h_i = \text{LM}_\phi(z_i, h_{<i}) \text{ (Eq. 1).}$$

The next-token distribution uses the last layer and a softmax: $p_\phi(z_{i+1} | h_{\leq i}) = \text{softmax}(W_\phi h_i^{(n)})$.

- **Fine-tuning objective:** maximize log likelihood of training y given x :

$$\max_{\phi} \log p_\phi(y | x) = \sum_{i \in Y_{\text{idx}}} \log p_\phi(z_i | h_{<i}). \text{ (Eq. 2)}$$

Fine-tuning updates ϕ .

- **Prefix-tuning method (core math):**
 - Introduce a trainable prefix matrix P_θ of shape $|P_{\text{idx}}| \times \dim(h_i)$.
 - Set

$$h_i = \begin{cases} P_\theta[i, :] & \text{if } i \in P_{\text{idx}} \\ \text{LM}_\phi(z_i, h_{<i}) & \text{otherwise} \end{cases} \text{ (Eq. 3)}$$

So the prefix simply *replaces* the activations in the left context for those prefix positions; other activations depend on it via recurrence. Crucially only θ (prefix) is trained while ϕ is frozen.

- Empirically they reparametrize to stabilize training: $P_\theta[i, :] = \text{MLP}_\theta(P'_\theta[i, :])$ (train a small P' plus an MLP) and save only P_θ after training.
- **Other logical/mathematical observations:**
 - Prefix length trades expressivity vs overfitting; performance grows up to a threshold (e.g., ~200 for summarization, ~10 for table-to-text) then slightly drops. This is shown empirically (plots/tables).
 - Embedding-only tuning (only tune token embeddings) is strictly less expressive and underperforms full prefix tuning. Infixing (putting trainable activations between x and y rather than before x) slightly underperforms prefixing.

Q1 — What does equation (1) $h_i = \text{LM}_\phi(z_i, h_{<i})$ mean? Explain components and how the next token probability is computed.

Equation (1) says that the activation vector h_i at timestep i is produced by the (pretrained) language model LM_ϕ given the current token z_i and the left context activations $h_{<i}$. Practically:

- h_i bundles activations from all transformer layers at position i .
 - The next token distribution uses the top (last) layer of h_i , denoted $h_i^{(n)}$, and a linear map W_ϕ :
 $p_\phi(z_{i+1} \mid h_{\leq i}) = \text{softmax}(W_\phi h_i^{(n)})$.
 - So prediction is a softmax of logits given by projecting the last-layer activation. This makes the model autoregressive: each next token depends on the computed activations from earlier tokens.
-

Q2 — Derive the fine-tuning training objective (Eq. 2) and explain why it updates ϕ .

For a dataset of input-output pairs (x, y) , the goal is maximize the conditional log-likelihood:

$$\log p_\phi(y \mid x) = \sum_{i \in Y_{\text{idx}}} \log p_\phi(z_i \mid h_{<i}),$$

because the model factorizes autoregressively over output positions. During fine-tuning, ϕ parametrizes LM_ϕ and therefore appears in every factor p_ϕ ; optimizing this objective by gradient descent updates ϕ to increase the likelihood of the training outputs. This is standard maximum-likelihood / cross-entropy training.

Q3 — In prefix-tuning, how does the training parameter set differ from fine-tuning? Use the equations to justify why only θ is updated.

In prefix-tuning Eq. (3) sets $h_i = P_\theta[i, :]$ when i is a prefix position and uses the frozen LM otherwise. The log-likelihood objective is the same as Eq. (2), but now only the prefix parameters θ appear as free parameters (because ϕ is held fixed). Although h_i for non-prefix positions depends on P_θ (through recurrence), the gradients flow only to θ ; ϕ remains unchanged. Thus optimization modifies θ to steer the frozen LM_ϕ .

Q4 — Why reparameterize $P_\theta[i, :] = \text{MLP}_\theta(P'_\theta[i, :])$? What is the mathematical rationale?

Directly optimizing the large matrix P_θ led the authors to unstable optimization and worse performance. Reparameterizing expresses the prefix rows as the output of a small-dimension latent P'_θ passed through a (larger) MLP. Mathematically this:

- reduces the number of directly optimized parameters (optimizing the smaller P'_θ),
 - provides an implicit low-dimensional manifold / regularization (the MLP maps a compact latent to the high-dim activation space),
 - often improves conditioning of gradients and reduces sensitivity to initialization and learning rate.
- After training, you can materialize P_θ and discard MLP internals if desired (save only prefix activations).
-

Q5 — Using the model notation, show how a prefix of length L affects the computation of h_k for a token k that lies right of the prefix.

Let prefix occupy positions $1 \dots L$. For any token at position $k > L$, its activation depends on all left context activations $h_{<k}$. Because $h_{1..L}$ are set to P_θ (Eq.3), they directly influence the attention inputs and thus every subsequent h_j for $j > L$. So mathematically:

$$h_k = \text{LM}_\phi(z_k, h_{<k}) \text{ and } h_{<k} \text{ includes } P_\theta[1:L, \cdot],$$

which means gradients of the loss w.r.t. parameters of P_θ are nonzero — the prefix steers downstream activations through the model recurrence/attention.

analyze document

Q6 — The paper measures that prefix-tuning uses ~0.1% of parameters and yet approaches fine-tuning performance. How can this be reconciled mathematically (intuition)?

A6 — Intrinsic dimension / expressive steering.

Two logical points justify this:

1. **Low intrinsic dimension:** recent work suggests fine-tuning moves the model in a low-dimensional effective subspace; thus a small set of parameters can approximate the needed direction. Prefix tuning exploits this by learning a compact prefix that, through the frozen LM, creates the required distributional shift. (Paper cites intrinsic-dimension ideas.)
 2. **Indirect control via activations:** a relatively small set of prefix activation vectors, when placed in the left context, is amplified through the LM's multi-layer attention and feed-forward layers — so small vectors can have large downstream effects. This is a nonlinear but powerful mechanism: the prefix doesn't need to encode full weights, it only needs to steer activations.
-

Q7 — Explain mathematically why embedding-only tuning is strictly less expressive than full prefix-tuning.

A7 — Explanation.

Embedding-only tuning optimizes token embeddings for a (short) sequence of pseudo-tokens; the transformer then computes upper activations from those embeddings. Full prefix-tuning however specifies activations at *all layers* for prefix positions (via P_θ which matches the dimension of h_i). Thus full prefix tuning can set information *at every hidden layer/time-step*, while embedding-only can only affect the bottom layer and must rely on the LM to transform that into upper layer effects. Hence embedding-only is an (expressive) subset — strictly weaker — and this matches the empirical drop in performance.

Q8 — The attention weight formula in the sample document was $\alpha_i^{(t)} = \frac{\exp(h_t^\top h_i)}{\sum_{j < t} \exp(h_t^\top h_j)}$. If a learned prefix increases h_i in the direction of h_t , what happens to $\alpha_i^{(t)}$ and downstream context vector c_t ?

(This uses the attention example from your sample doc as a pattern.)

sample

A8 — Effect on attention.

If h_i becomes more aligned with h_t (dot product increases), then $\exp(h_t^\top h_i)$ increases relative to other terms: $\alpha_i^{(t)}$ increases. The context vector $c_t = \sum_{i < t} \alpha_i^{(t)} h_i$ will therefore weight that h_i more and move c_t towards h_i . So a prefix that nudges certain left-context activations to align with target generation points will bias attention and thus influence token probability via $h \rightarrow$ logits. (This shows how a prefix can steer attention mathematically.)

sample

Q9 — Why does increasing prefix length help up to a point and then hurt (empirical observation)? Provide a mathematical/learning perspective.

A9 — Bias-variance / capacity reasoning.

- **Increasing length** increases trainable parameters $\rightarrow$ higher representation capacity, enabling better fit to the training distribution (reduced bias).
- **Beyond threshold** the model can overfit the training data: training loss continues to drop but generalization (test metrics) slightly degrades (higher variance). This matches the paper's observed curve: performance grows with prefix length until a sweet-spot, then drops. Formally this is standard capacity vs generalization tradeoff. Empirical thresholds (~ 200 for summarization; ~ 10 for table-to-text) reflect dataset complexity and input length

Q: Given the autoregressive model notation $h_i = \text{LM}_\phi(z_i, h_{<i})$ and $p_\phi(z_{i+1} \mid h_{\leq i}) = \text{softmax}(W_\phi h_i^{(n)})$, explain what each symbol means: h_i , LM_ϕ , z_i , $h_i^{(n)}$, and W_ϕ .

Solution (step-by-step):

1. z_i : token at position i in the concatenated sequence $z = [x; y]$ (could be input or output token).
2. h_i : the activation vector at position i ; often stacks activations across layers at that position, dimensionality d .
3. LM_ϕ : pretrained language model function parameterized by weights ϕ , mapping current token + left context to activation.
4. $h_i^{(n)}$: the last-layer (top) activation at position i used for next-token logits.

5. W_ϕ : linear projection matrix (part of LM parameters) that maps last-layer activations to vocabulary logits before softmax.

Problem 2 — Factorization of likelihood

Q: Show how the conditional likelihood $\log p_\phi(y \mid x)$ factors autoregressively into a sum over token log-probabilities.

Solution:

1. By autoregressive factorization, $p_\phi(y \mid x) = \prod_{i \in Y_{\text{idx}}} p_\phi(z_i \mid z_{<i})$.
2. Take log: $\log p_\phi(y \mid x) = \sum_{i \in Y_{\text{idx}}} \log p_\phi(z_i \mid z_{<i})$.
3. Replace $z_{<i}$ dependence by $h_{<i}$ if desired, yielding same sum. This gives Eq. (2) style expression.

Problem 3 — Prefix insertion equation

Q: Given prefix matrix P_θ and Eq. $h_i = P_\theta[i, :]$ if $i \in P_{\text{idx}}$ else $\text{LM}_\phi(z_i, h_{<i})$, show how this modifies the context for a token at position $k > L$.

Solution:

1. Let prefix occupy positions $1..L$. For $k > L$, $h_{<k}$ contains $h_1, \dots, h_L$ which equal $P_\theta[1..L, :]$.
2. So $h_k = \text{LM}_\phi(z_k, [P_\theta[1..L, :], h_{L+1..k-1}])$.
3. Thus prefix values enter attention and feed-forward computations for h_k , affecting its value and subsequent predictions.

Problem 4 — Gradient non-zero proof (short)

Q: Prove $\partial L / \partial P_\theta \neq 0$ in general for loss L that depends on model outputs to the right of the prefix.

Solution:

1. Loss depends on outputs at positions $i > L$: $L = f(h_{\leq T})$.
2. By chain rule: $\partial L / \partial P_\theta = \sum_j \partial L / \partial h_j \cdot \partial h_j / \partial P_\theta$.
3. For some $j > L$, $\partial L / \partial h_j \neq 0$. Also $\partial h_j / \partial P_\theta \neq 0$ because h_j computed via attention over earlier h which include prefix entries. Thus product nonzero; sum nonzero in general.

Problem 5 — Reparameterization motivation

Q: Mathematically argue why using $P_\theta[i, :] = \text{MLP}_\theta(P'_\theta[i, :])$ helps optimization.

Solution (steps):

1. Directly optimizing a large P_θ is optimizing in a high-dimensional space — may have poor conditioning.
 2. Reparameterizing maps a low-dim latent P'_θ through an MLP: this reduces number of free variables and imposes a smooth manifold.
 3. Gradients backprop through MLP; small changes in P'_θ yield smoother, controlled updates in activation space, improving stability.
 4. Empirically this reduces optimization variance and improves generalization.
-

Problem 6 — Attention alignment effect

Q: Given attention weights $\alpha_i^{(t)} = \frac{\exp(h_t^\top h_i)}{\sum_{j < t} \exp(h_t^\top h_j)}$, show how increasing alignment $h_t^\top h_i$ by $\Delta > 0$ changes $\alpha_i^{(t)}$.

Solution:

1. Original numerator $N = \exp(h_t^\top h_i)$. New numerator $N' = \exp(h_t^\top h_i + \Delta) = \exp(\Delta)N$.
 2. Denote denominator $D = \sum_{j < t} \exp(h_t^\top h_j)$. New denominator $D' = D - N + N' = D + (\exp(\Delta) - 1)N$.
 3. New weight $\alpha'_i = \frac{\exp(\Delta)N}{D + (\exp(\Delta) - 1)N}$.
 4. Since $\exp(\Delta) > 1$, $\alpha'_i > \frac{N}{D} = \alpha_i$. So alignment increase raises attention weight.
-

Problem 7 — Parameter fraction arithmetic

Q: If LM has $M = 1,500,000,000$ parameters (1.5B) and prefix stores $p = 750,000$ parameters, compute the stored-parameters percent. Show digit-by-digit arithmetic.

Solution (digit-by-digit):

1. Fraction $f = \frac{750,000}{1,500,000,000}$.
2. Cancel three zeros numerator and denominator: $\frac{750}{1,500,000}$.
3. Cancel another three zeros: $\frac{0.75}{1,500}$ but better to divide: compute $1,500,000/750 = 2000$ so inverse: $750/1,500,000 = 1/2000$.
4. So $f = 1/2000 = 0.0005$.
5. Percentage = $0.0005 \times 100 = 0.05\%$.

Problem 8 — Expressivity argument (embedding-only vs prefix)

Q: Using layer notation, explain why specifying activations at all layers is more expressive than only changing input embeddings.

Solution:

1. Input-embedding tuning modifies only bottom-layer inputs $h^{(1)}$ indirectly; the LM must transform these via fixed layers to upper layers.
2. Prefix P_θ can set activations for every layer/time position — effectively controlling $h^{(1)}, h^{(2)}, \dots, h^{(n)}$ at prefix positions.
3. Therefore any effect achievable by embedding-only can be simulated by a carefully chosen prefix (via propagation), but not conversely — prefix has strictly larger representational set.

Problem 9 — Chain rule for dependency

Q: Write the chain-rule expression for $\partial L / \partial P_\theta$ explicitly summing layers and time steps.

Solution:

1. Let $h_t^{(\ell)}$ denote layer ℓ activation. Then

$$\frac{\partial L}{\partial P_\theta} = \sum_t \sum_\ell \frac{\partial L}{\partial h_t^{(\ell)}} \frac{\partial h_t^{(\ell)}}{\partial P_\theta}.$$

2. Each $\frac{\partial h_t^{(\ell)}}{\partial P_\theta}$ captures how prefix activations influence that specific layer and timestep via attention and residuals.
3. This explicit sum clarifies gradient flow paths.

Problem 10 — Softmax sensitivity

Q: For logits vector s with softmax $\sigma(s)$, show how increasing a single logit s_k by δ changes σ_k . Provide formula.

Solution:

1. Original $\sigma_k = \frac{e^{s_k}}{\sum_j e^{s_j}}$.
2. New $s'_k = s_k + \delta$. New numerator $e^{s_k + \delta} = e^\delta e^{s_k}$. New denominator $D' = \sum_{j \neq k} e^{s_j} + e^\delta e^{s_k}$.
3. So $\sigma'_k = \frac{e^\delta e^{s_k}}{D'}$. No closed linear form but can write:

$$\sigma'_k = \frac{e^\delta \sigma_k}{1 + (e^\delta - 1)\sigma_k}.$$

4. This displays saturating effect: large δ pushes $\sigma'_k \rightarrow 1$.

Problem 11 — Toy gradient calculation

Q: Suppose a single output logit $u = w^\top h$ with loss $L = -\log \text{softmax}(u)_y$. Derive $\partial L / \partial h$.

Solution:

1. For vocabulary size 2 toy or general: softmax derivative known. Let $\hat{p} = \text{softmax}(u)$.
2. For class y , loss $L = -\log \hat{p}_y$.
3. $\partial L / \partial u_j = \hat{p}_j - \mathbf{1}_{j=y}$.
4. Since $u_j = w_j^\top h$, $\partial u_j / \partial h = w_j$.
5. So $\partial L / \partial h = \sum_j (\hat{p}_j - \mathbf{1}_{j=y}) w_j$.
6. Compact: $W^\top (\hat{p} - e_y)$.

Problem 12 — Prefix length tradeoff (qualitative math)

Q: Provide a short mathematical explanation (bias-variance) why increasing prefix length helps then hurts performance.

Solution (steps):

1. Increasing length increases model capacity $C(L)$ monotone in L .
2. Training error decreases with C (bias $\downarrow$). But generalization error = training error + variance(L).
3. Variance typically increases with capacity, so test error = bias(L) + variance(L) is U-shaped.
4. So optimal L^* balances bias and variance; beyond that, overfitting causes test performance drop.

Problem 13 — Numerical check: softmax change example

Q: Numerically compute softmax change if a logit vector is $[1, 2, 3]$ and we increase the first logit by 2 (to 3). Compute original and new softmax for first position (digit-by-digit).

Solution (digit-by-digit):

1. Original logits: $s = [1, 2, 3]$. Compute exponentials:

- $e^1 \approx 2.718281828$
 - $e^2 \approx 7.389056099$
 - $e^3 \approx 20.08553692$
2. Denominator $D = 2.718281828 + 7.389056099 + 20.08553692 =$
- Add first two: $2.718281828 + 7.389056099 = 10.107337927$.
 - Add third: $10.107337927 + 20.08553692 = 30.192874847$.
3. Original $\sigma_1 = 2.718281828/30.192874847$.
- Divide: 30.192874847 into 2.718281828. Compute ratio:
 - $30.192874847 \times 0.09 = 2.71735873623$ (close). Try 0.090003:
 $30.192874847 * 0.090003 = 2.718268 \sim$ close enough.
 - So $\sigma_1 \approx 0.0900$ ($\approx 9.00\%$).
4. New logits: [3,2,3]. Exponentials:
- $e^3 \approx 20.08553692$ (for first)
 - $e^2 \approx 7.389056099$
 - $e^3 \approx 20.08553692$
5. New denominator $D' = 20.08553692 + 7.389056099 + 20.08553692 = 47.560129939$.
6. New $\sigma'_1 = 20.08553692/47.560129939$.
- Divide: $47.560129939 \times 0.422 = 20.053$. Try 0.4221: $47.560129939 * 0.4221 = 20.0588$. Try 0.4223: $47.560129939 * 0.4223 = 20.067$. Sufficient: $\sigma'_1 \approx 0.422$ ($\sim 42.2\%$).
7. So σ_1 changed from $\approx 9.0\%$ to $\approx 42.2\%$ after increasing first logit by 2.

Problem 14 — Derive effect of prefix on logits

Q: Assume last-layer activation for position i is $h_i^{(n)}$. If a prefix causes $h_i^{(n)}$ to change by Δh , write first-order Taylor approximation for change in logits $s = W_\phi h_i^{(n)}$.

Solution:

1. Original logits $s = W_\phi h$.
2. New activation $h' = h + \Delta h$.
3. New logits $s' = W_\phi(h + \Delta h) = s + W_\phi \Delta h$.

4. So change in logits $\approx W_\phi \Delta h$. Softmax change follows from previous softmax sensitivity.
-

Problem 15 — Low intrinsic dimension argument (short proof sketch)

Q: Give a concise mathematical argument why a small prefix vector set can emulate a low-dimensional fine-tuning update.

Solution (sketch):

1. Suppose fine-tuning updates weights by $\Delta\phi$ lying roughly in a low-dimensional subspace U of dimension k .
 2. The effect of $\Delta\phi$ on activations for new inputs can be approximated by a linear map A applied to $\Delta\phi$: $\Delta h \approx A\Delta\phi$.
 3. If U is small, then Δh lies in low-dim image $A(U)$. Prefix P_θ can be optimized to produce activations in that same low-dim subspace directly, thus simulating the effect of $\Delta\phi$ without changing ϕ .
 4. Hence small number of prefix parameters suffice.
-

Problem 16 — Multi-task storage math

Q: If each task stores a prefix of 250K params and you have 40 tasks, compute total stored params and percent of a 2B-parameter model. Show steps.

Solution (step-by-step):

1. Per-task prefix = 250,000. For 40 tasks: total = $250,000 \times 40$.
 2. Multiply: $25,0000 \times 40 = 10,000,000$ (since $250,000 = 25 \times 10^4$, but do exact):
 - $250,000 \times 10 = 2,500,000$.
 - Times 4 = $2,500,000 \times 4 = 10,000,000$.
 3. So total stored = 10,000,000 parameters.
 4. Model size $M = 2,000,000,000$. Fraction $f = 10,000,000 / 2,000,000,000$.
 5. Cancel six zeros: $10 / 2000 = 0.005$.
 6. Percentage = $0.005 \times 100 = 0.5\%$.
-

Problem 17 — Infix vs prefix comparison (logical)

Q: Give a short derivation/argument why inserting trainable vectors *between* x and y (infixing) may be less effective than prefixing before x .

Solution:

1. Prefixing sits in the left context before both x and y — it can influence how x is processed (conditioning) and how y is generated.
2. Infixing (between x and y) cannot affect encoding of x ; it only affects y -generation given x . Some tasks benefit from conditioning x 's internal representation; removing that capacity reduces expressivity.
3. Thus prefixing has at least the expressive power of infixing and can be strictly better for tasks needing to adjust input encoding.

Problem 18 — Gradients through attention (short derivation)

Q: For scaled dot-product attention $\text{Att}(Q, K, V) = \text{softmax}(QK^\top / \sqrt{d_k})V$, show qualitatively how changing a prefix K row impacts output.

Solution (steps):

1. Changing a prefix row in K alters the dot-products $QK^\top$ for queries Q attending to prefix positions.
2. These dot-products change attention weights via softmax, and thus change weighted sum over rows of V .
3. So prefix changes in K directly modify attention outputs and hence downstream activations. Gradient flows from loss to attention weights to K entries, making prefix trainable.

Problem 19 — Construct an exam question: gradient magnitude intuition

Q (student): Why might gradients w.r.t. prefix entries be smaller or larger than gradients w.r.t. LM weights? Provide at least two factors.

Solution (answer points):

1. **Scale & depth:** Prefix entries are early in the computation graph; they influence all downstream layers—gradients accumulate through many residual connections—could cause vanishing or amplification depending on network depth and normalization.
2. **Parameterization & size:** Prefix is small; per-parameter gradient magnitudes may be larger to achieve same global effect. Conversely, LM weights are many and distributed; each weight may receive smaller gradient contributions.
3. **Reparametrization (MLP):** If prefix uses an MLP mapping, gradients pass through MLP which may attenuate or amplify them depending on MLP initialization & activation functions.

Problem 20 — Show equivalence condition (conceptual)

Q: Under what (strong) condition could prefix-tuning exactly replicate a particular fine-tuned model's outputs for all inputs? Describe idea mathematically.

Solution (steps):

1. For prefix-tuning to replicate fine-tuned LM $\phi' = \phi + \Delta\phi$, there must exist P_θ such that for all x , model outputs with frozen ϕ and prefix P_θ equal outputs of LM with ϕ' and no prefix:

$$p_{\phi, P_\theta}(y | x) = p_{\phi'}(y | x) \forall x, y.$$

2. This requires that the effect of $\Delta\phi$ on activations is realizable by setting left-context activations $h_{1..L}$ appropriately for all x .
 3. This is only possible if $\Delta\phi$ induces activations in the image of the map that prefixes can set — effectively a very strong constraint; in practice unlikely for all inputs, but possible approximately for a task-limited input distribution.
-

Problem 21 — Design problem (constructive)

Q: Design a small experiment to test whether prefix tuning helps attention to focus on particular tokens. Briefly state loss, metric, and hypothesis.

Solution (steps):

1. **Dataset:** synthetic task where correct output depends strongly on one token in input (e.g., copy index).
 2. **Models:** frozen LM + random prefix initialized vs. no-prefix baseline.
 3. **Training:** train prefix to minimize cross-entropy predicting output.
 4. **Metric:** attention mass on the relevant input token measured as average attention weight to that token during generation.
 5. **Hypothesis:** trained prefix increases attention mass on the relevant token compared to baseline, showing prefix steers attention.
-

Problem 22 — Long answer derivation: effect on perplexity (sketch)

Q: Sketch how changing prefix P_θ could reduce perplexity on a target dataset without updating ϕ . Write the relation between perplexity and log-likelihood and explain.

Solution (steps):

1. Perplexity per token defined as $\text{PPL} = \exp(-\frac{1}{N} \sum_{i=1}^N \log p(z_i | z_{<i}))$.
2. Training prefix maximizes $\sum \log p(z_i | z_{<i})$ over dataset; increasing this sum decreases the exponent's negative term, thereby reducing PPL.

3. Mathematically prefix changes activations $h_{<i} \rightarrow$ changes logits $\rightarrow$ increases correct-token log-probabilities.
 4. So even with ϕ fixed, optimizing P_θ improves conditional probabilities on the target distribution, yielding lower perplexity.
-

TRANSFORMER

Q1.1 Unified Text-to-Text Framework

Problem:

How does T5 generalize all NLP tasks into a single modeling objective?

Solution:

Every NLP task is converted into a **sequence-to-sequence mapping**:

$$x_{\text{input}} \rightarrow y_{\text{target}}$$

Examples:

- Translation: “translate English to German: That is good.” $\rightarrow$ “Das ist gut.”
- Classification: “cola sentence: The course is jumping well.” $\rightarrow$ “not acceptable”

This unified *text-to-text* paradigm allows the same **model**, **loss**, and **optimization** for all tasks, simplifying transfer learning.

Q1.2 Transformer Core Equation

Problem:

Derive the self-attention computation used within the Transformer architecture.

Solution:

Given queries Q , keys K , and values V :

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

Each output vector attends to all previous tokens (for encoder) or past tokens only (for decoder via causal masking).

Q1.3 Multi-Head Attention Composition

Problem:

Why is multi-head attention more expressive than single-head?

Solution:

$$\begin{aligned} \text{MHA}(Q, K, V) &= \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O \\ \text{head}_i &= \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \end{aligned}$$

Each head learns different subspace projections $W_i^{Q,K,V}$, enabling **parallel contextual reasoning**.

Q1.4 Encoder-Decoder Mapping**Problem:**

Explain mathematically how encoder and decoder interact.

Solution:

For encoder hidden states H_e :

$$H_e = \text{Encoder}(X)$$

The decoder predicts autoregressively:

$$p(y_t | y_{<t}, X) = \text{softmax}(W_o h_t)$$

where $h_t = \text{Decoder}(y_{<t}, H_e)$.

The loss is the **negative log-likelihood**:

$$L = - \sum_t \log p(y_t | y_{<t}, X)$$

Q1.5 Positional Encoding**Problem:**

How does T5 handle positional information efficiently?

Solution:

Instead of sinusoidal encodings, T5 uses **relative position embeddings**:

$$a_{ij} = \frac{Q_i K_j^\top}{\sqrt{d_k}} + r_{|i-j|}$$

where $r_{|i-j|}$ is a learned scalar bias depending on token distance.
This improves generalization across long sequences.

Q1.6 Layer Normalization Variant

Problem:

What is the modification to LayerNorm in T5?

Solution:

T5 removes bias and applies normalization *outside* the residual path:

$$\text{LN}(x) = \frac{x - \mu(x)}{\sigma(x)}$$
$$y = x + \text{SubLayer}(\text{LN}(x))$$

This stabilizes very deep models (up to 11B parameters).

Q1.7 Denoising Objective Function

Problem:

Describe the denoising (masked-span) objective used for pre-training.

Solution:

Given input sequence $x = (x_1, \dots, x_T)$,
random spans are replaced by **sentinel tokens** $\langle X_i \rangle$.

$$L_{\text{denoise}} = - \sum_{t \in \text{masked}} \log P_{\theta}(x_t | x_{\setminus \text{masked}})$$

The target is the concatenation of missing spans, delimited by sentinels.

Example:

Input $\rightarrow$ "Thank $\langle X \rangle$ inviting me $\langle Y \rangle$ party."

Target $\rightarrow$ " $\langle X \rangle$ you for $\langle Y \rangle$ to your $\langle Z \rangle$ "

Q1.8 Learning Rate Schedule

Problem:

Show the formula for the inverse square-root learning rate used in T5.

Solution:

$$\eta_n = \eta_0 \times \frac{1}{\sqrt{\max(n, k)}}$$

where

- n = current step
- $k = 10^4$ warm-up steps
- $\eta_0 = 0.01$

Q1.9 Pre-training Data Scale

Problem:

How is the total number of tokens for pre-training computed?

Solution:

Given

- steps $S = 2^{19} = 524,288$
- batch tokens $B = 2^{16} = 65,536$,

$$\text{Total Tokens} = S \times B = 2^{35} \approx 34 \text{ billion tokens.}$$

Q1.10 Loss for Masked Span Reconstruction

Problem:

Formulate T5's cross-entropy loss for masked tokens.

Solution:

$$L = - \sum_{i=1}^N \sum_t \log P_{\theta}(y_t^{(i)} | y_{<t}^{(i)}, x^{(i)})$$

This **teacher-forced** training minimizes prediction error across masked text spans.

Q1.11 Evaluation Metric (Translation)

Problem:

What metric evaluates translation performance in T5 experiments?

Solution:

$$\text{BLEU} = BP \times \exp\left(\sum_{n=1}^4 w_n \log p_n\right)$$

where:

- BP : brevity penalty
- p_n : n-gram precision
- $w_n = \frac{1}{4}$

T5 reports BLEU for English–German, French, Romanian tasks.

Q1.12 ROUGE for Summarization

Problem:

Define ROUGE-2-F score.

Solution:

$$\text{ROUGE-2-F} = \frac{2 \times P_2 \times R_2}{P_2 + R_2}$$

where P_2 = bigram precision, R_2 = bigram recall.

It measures overlap between generated and reference summaries.

Q1.13 Sparse Attention Efficiency

Problem:

How does sparse attention reduce computational cost?

Solution:

Standard complexity: $O(T^2)$

Sparse/linearized version:

$$\text{Attn}(Q, K, V) \approx Q(\phi(K)^T V)$$

where $\phi(\cdot)$ is a kernel map,

reducing cost to $O(T)$ or $O(T \log T)$.

Q1.14 Model Scaling

Problem:

How does increasing model size affect performance?

Solution:

Empirically observed **scaling law**:

$$L_{\text{val}} \propto N^{-\alpha}$$

where N = parameter count, $\alpha \in [0.05, 0.1]$.

Thus, doubling parameters yields diminishing but consistent returns.

Q1.15 Parameter Sharing Impact

Problem:

Compare shared vs. non-shared encoder–decoder parameters.

Solution:

Model	Params	GLUE	CNNDM	EnDe
Encoder–Decoder	2P	83.28	19.24	26.98
Shared Parameters	P	82.81	18.78	26.72

Parameter sharing reduces memory without large performance loss.

Q1.16 Dropout Regularization

Problem:

Formulate how dropout applies in T5.

Solution:

At layer outputs:

$$\begin{aligned} h' &= \text{Dropout}(h, p) \\ h_{\text{final}} &= h' + \text{Residual}(h) \end{aligned}$$

Drop probability $p = 0.1$

applied to feed-forward layers, attention weights, and embeddings.

Q1.17 Denoising vs. Language Modeling

Problem:

Why does denoising outperform autoregressive LM?

Solution:

Autoregressive:

$$L_{\text{LM}} = - \sum_t \log P(w_t | w_{<t})$$

Denoising:

$$L_{\text{Denoise}} = - \sum_{t \in \mathcal{M}} \log P(w_t | w_{\setminus \mathcal{M}})$$

Denoising learns **bidirectional context**, leading to richer representations.

Q1.18 Fine-tuning Optimization**Problem:**

State the optimization settings for fine-tuning T5.

Solution:

Learning rate = 0.001, Steps = $2^{18} = 262,144$
 Batch size = 128, Sequence length = 512

The model checkpoint with highest validation accuracy is selected.

Q1.19 Data Cleaning Heuristics**Problem:**

Describe the filtering rules for the C4 dataset mathematically.

Solution:

Let L_i be a text line.

Keep L_i if:

$$\begin{cases} |L_i| \geq 5 \text{ words} \\ L_i \text{ ends with } [.!?"'] \\ \text{lang}(L_i) = \text{English (p} > 0.99) \\ L_i \notin \text{duplicates or banned list} \end{cases}$$

Total clean corpus $\approx$ 750 GB text.

Q1.20 Pre-training Token Masking Ratio

Problem:

What percentage of tokens are masked in T5 pre-training?

Solution:

$$p_{\text{mask}} = 0.15$$

i.e., 15% of tokens replaced by sentinel markers.

Consecutive masked spans are merged into one sentinel per span.

Q1.21 Objective Unification

Problem:

Show how all task losses share the same maximum-likelihood form.

Solution:

For all tasks $\mathcal{T}$:

$$L_{\mathcal{T}}(\theta) = - \sum_{(x,y) \in \mathcal{D}_{\mathcal{T}}} \log P_{\theta}(y \mid x)$$

Thus translation, summarization, classification, and QA share identical training dynamics — only inputs differ.

Q1.22 Example: Span Corruption Encoding

Problem:

Given input *"You invited me to your party last week."*
simulate corruption span indices [3,4] and [7].

Solution:

Input $\rightarrow$ "You <X> to your party <Y>."

Target $\rightarrow$ "<X> invited me <Y> last <Z>"

This preserves token order while forcing semantic reconstruction.

LORA

Q1.1 Core idea

Problem: Write the core low-rank adaptation formula and explain each term.

Solution:

LoRA represents the learned update as a low-rank factorization:

$$\Delta W = AB$$

so the adapted weight is

$$W' = W + \Delta W = W + AB,$$

where $W \in \mathbb{R}^{d \times k}$, $A \in \mathbb{R}^{d \times r}$, $B \in \mathbb{R}^{r \times k}$, and $r \ll \min(d, k)$.
 A and B are trainable; W is frozen.

LoRA_ChatGPT (1)

Q1.2 Parameter-count reduction

Problem: Compare the number of trainable parameters in full fine-tuning vs LoRA.

Solution:

- Full fine-tuning: train $d \times k$ parameters.
- LoRA: train $d \cdot r + r \cdot k = r(d + k)$ parameters.

Parameter reduction factor approximately:

$$\frac{r(d + k)}{dk} \approx \frac{r}{\min(d, k)} \text{ (for large } d, k)$$

So with $r \ll \min(d, k)$, huge savings.

LoRA_ChatGPT (1)

Q1.3 Forward-pass math

Problem: Show the forward-pass with LoRA inserted into a linear layer $Y = WX$.

Solution:

With LoRA:

$$Y = (W + AB)X = WX + A(BX).$$

Compute $u = BX$ (size $r \times n$), then Au (size $d \times n$), add to base WX .

LoRA_ChatGPT (1)

Q1.4 Initialization for zero initial effect

Problem: How to initialize A, B so the model behaves identically before training?

Solution:

Set

$$A \leftarrow 0, B \sim \mathcal{N}(0, \sigma^2)$$

so initially $\Delta W = AB = 0$ and $W' = W$. Commonly σ is small (e.g. 0.01).

LoRA_ChatGPT (1)

Q1.5 Scaling factor (alpha)

Problem: Explain LoRA's scaling α and how it's applied.

Solution:

LoRA often scales the low-rank update by $\frac{\alpha}{r}$:

$$\Delta W = \frac{\alpha}{r} AB.$$

Thus effective update used in forward:

$$Y = (W + \frac{\alpha}{r} AB)X.$$

Scaling controls update magnitude relative to base W .

LoRA_ChatGPT (1)

Q1.6 Backprop: gradients w.r.t. A and B

Problem: Derive $\frac{\partial L}{\partial A}$ and $\frac{\partial L}{\partial B}$ given $\frac{\partial L}{\partial \Delta W}$.

Solution:

By chain rule, treating $\Delta W = AB$:

$$\frac{\partial L}{\partial A} = \frac{\partial L}{\partial \Delta W} B^\top, \frac{\partial L}{\partial B} = A^\top \frac{\partial L}{\partial \Delta W}.$$

This matches teacher notes: A learns output directions, B learns input combinations.

LoRA_ChatGPT (1)

Q1.7 Parameter update step

Problem: Give the SGD/Adam update rules for A, B .

Solution:

Using basic gradient descent with learning rate η :

$$A \leftarrow A - \eta \frac{\partial L}{\partial A}, B \leftarrow B - \eta \frac{\partial L}{\partial B}.$$

With Adam, same gradient applied inside Adam's adaptive moment steps (moment, variance, bias-correction).

LoRA_ChatGPT (1)

Q1.8 Inference merging

Problem: Show how to merge LoRA weights into base weights for inference.

Solution:

Merged weight:

$$W' = W + \frac{\alpha}{r} AB.$$

If merged, the forward becomes standard $Y = W'X$; merging reduces runtime overhead if modular switching is not needed.

LoRA_ChatGPT (1)

Q1.9 Memory & compute during training

Problem: Compare memory/compute characteristics of LoRA vs full fine-tuning.

Solution:

- Memory: only store gradients/optimizer state for A and $B \rightarrow O(r(d + k))$ instead of $O(dk)$.
- Compute: extra two small matmuls per layer: compute $BX(r \times k \text{ or } r \times n)$ and $A(BX)(d \times r)$. Cost is $O(ndr + nrk)$, but with r small this is minor versus base WX .

LoRA_ChatGPT (1)

Q1.10 Example numeric sizes

Problem: For $W \in \mathbb{R}^{4096 \times 4096}$ and $r = 8$, compute trainable params and compare.

Solution:

Full: $4096^2 = 16,777,216$ params.

LoRA: $r(d + k) = 8(4096 + 4096) = 8 \times 8192 = 65,536$ params.

Reduction $\approx 256 \times$ fewer trainable params.

LoRA_ChatGPT (1)

Q1.11 Low-rank intuition (SVD link)

Problem: Explain LoRA as a low-rank approximation in relation to SVD.

Solution:

An SVD of an update ΔW is $\Delta W = U \Sigma V^T$. Truncating to rank r gives low-rank approximation $\widetilde{\Delta W} = U_r \Sigma_r V_r^T$, which can be represented as AB by setting $A = U_r \Sigma_r$, $B = V_r^T$. LoRA learns such a low-rank correction directly.

LoRA_ChatGPT (1)

Q1.12 Choice of rank r

Problem: What trade-offs govern choosing r ?

Solution:

- Larger $r \rightarrow$ more expressivity, more parameters and compute.
- Smaller $r \rightarrow$ smaller memory and faster, but may underfit task-specific adjustments. Pick r so $r(d + k)$ is acceptable while capturing major update directions. Empirically small $r(4-64)$ often suffices.

LoRA_ChatGPT (1)

Q1.13 Why freeze base W ?

Problem: Explain why LoRA keeps W frozen.

Solution:

Freezing W preserves pretrained knowledge, reduces memory/compute for training (no gradients for W), and isolates task-specific updates in small matrices A, B , enabling modular multi-task adapters.

LoRA_ChatGPT (1)

Q1.14 LoRA gradient flow intuition

Problem: Interpret $\frac{\partial L}{\partial A} = \frac{\partial L}{\partial \Delta W} B^\top$ intuitively.

Solution:

$\frac{\partial L}{\partial \Delta W}$ gives how each output weight entry affects loss; multiplying by $B^\top$ projects these signals into the low-rank output-basis learned by B , so A learns adjustments along those directions. Symmetrically for B .

LoRA_ChatGPT (1)

Q1.15 LoRA in code: forward pass (matrices & scaling)

Problem: Express the PyTorch-style forward computation including scaling.

Solution:

Given input x (batch):

```
delta = (A @ B) * (alpha / r) # shape: d x k
```

```
output = x @ (W.T + delta.T) # or output = W(x) + A(Bx) * (alpha/r)
```

Equivalently compute $u = Bx$, then output increment Au scaled by α/r .

LoRA_ChatGPT (1)

Q1.16 Combining LoRA modules

Problem: If multiple LoRA adapters (for tasks) exist, how to switch between them at inference?

Solution:

Keep W frozen and keep separate pairs (A_i, B_i) . For task i use:

$$W'_i = W + \frac{\alpha}{r} A_i B_i.$$

Switching is constant-time addition of the small $A_i B_i$ (or merge and reload weights). This enables modular task-switching.

Q1.17 Stability and warm start

Problem: Why zero-init A improves stability on fine-tuning?

Solution:

Zero-init ensures initial model behavior identical to pretrained W , preventing sudden large changes in outputs and gradients. This produces stable training dynamics and safer adaptation.

Q1.18 Combined update with optimizer (Adam)

Problem: Write Adam's moment update sketch for A .

Solution:

Let $g_t = \nabla_A L$. Adam steps:

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \\ A &\leftarrow A - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \end{aligned}$$

Similarly for B . Adam adapts step size per parameter of A, B .

Q1.1 Parameter-Efficient Fine-Tuning

Full fine-tuning updates all parameters Φ , while LoRA learns a compact parameter set Θ :

$$\max_{\Theta} \sum_{(x,y) \in Z} \sum_{t=1}^{|y|} \log P_{\Phi_0 + \Delta\Phi(\Theta)}(y_t \mid x, y_{<t})$$

with $|\Theta| \ll |\Phi_0|$.

This drastically reduces the number of trainable parameters—often to 0.01% of the original model size.

Q1.2 Low-Rank Decomposition

Instead of directly updating weights $W_0 \in \mathbb{R}^{d \times k}$, LoRA constrains updates as:

$$\Delta W = BA, B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times k}, r \ll \min(d, k)$$

Hence,

$$W = W_0 + \Delta W = W_0 + BA$$

This reduces the update rank from d to r , imposing a low-rank adaptation manifold on the weight space.

Q1.3 Forward Computation under LoRA

During adaptation, the modified linear transformation is:

$$h = W_0x + \Delta Wx = W_0x + BAx$$

where W_0 is frozen.

If initialized with $B = 0$ and $A \sim \mathcal{N}(0, \sigma^2)$, the initial update $\Delta W = 0$, ensuring stable optimization.

Q1.4 Scaling Factor and Initialization

To stabilize learning, LoRA rescales updates as:

$$\hat{h} = W_0x + \frac{\alpha}{r}BAx$$

where α is a scaling constant (typically = rank r).

This ensures gradient magnitude consistency across different ranks.

Q1.5 Optimization Efficiency

Since W_0 is frozen, gradients are computed only for A, B :

$$\nabla_{A,B}L = \frac{\partial L}{\partial h} \cdot \frac{\partial h}{\partial (A,B)}$$

Memory and optimizer state savings $\approx 3\times$, since only $O(r(d+k))$ parameters are updated versus $O(dk)$ in full fine-tuning.

Q1.6 Expressivity and Convergence

When LoRA rank $r \rightarrow \text{rank}(W_0)$,

LoRA $\rightarrow$ Full Fine-Tuning

Hence, LoRA generalizes fine-tuning while enabling trade-offs between compute and expressivity.

Q1.7 Application to Transformer Attention

In the Transformer, LoRA applies to specific projection matrices (W_q, W_k, W_v, W_o):

$$|\Theta| = 2 \times \hat{L}_{LoRA} \times d_{model} \times r$$

Usually, only W_q and W_v are adapted for optimal efficiency, balancing expressivity and parameter count.

Q1.8 Intrinsic Rank & Empirical Finding

Empirically, LoRA achieves competitive performance even with $r = 1$ or 2 , indicating that the update matrix ΔW lies on a low intrinsic-rank manifold.

$$\text{Rank}(\Delta W) \ll \text{Rank}(W_0)$$

Thus, most adaptation information resides in a very small subspace.

Q1.9 Subspace Similarity (Grassmann Distance)

To measure subspace overlap between adaptations of different ranks r_1, r_2 :

$$\phi(A_{r_1}, A_{r_2}, i, j) = \frac{\|U_{r_1}^{i\top} U_{r_2}^j\|_F^2}{\min(i, j)} \in [0, 1]$$

High ϕ indicates similar principal subspaces — showing that LoRA's top singular directions remain stable across ranks.

Q1.10 Relation between Pre-trained and Adapted Weights

Projecting W onto ΔW 's subspace:

$$\|U^\top W V^\top\|_F$$

reveals that ΔW correlates with W but amplifies underrepresented directions, enhancing task-specific representations without overwriting pre-trained structure.

Q1.11 Memory & Speed Comparison

For a model with $d_{model} = 12288$, $r = 4$:

$$\frac{|\Theta_{LoRA}|}{|\Phi_{Full}|} \approx 10^{-4}$$

GPU memory drops from 1.2 TB $\rightarrow$ 350 GB ($\approx 3\times$ improvement).
Training throughput increases by 25%, as frozen weights avoid gradient updates.

Q1.12 Rank-Performance Trade-off

Empirically:

Rank r Accuracy (WikiSQL) Accuracy (MNLI)

1	70.4	90.7
2	73.4	91.4
4-8	73.8-74.0	91.6-91.7

Thus, LoRA achieves saturation in performance for $r \leq 8$, proving diminishing returns beyond that.

Q1.13 Inference Efficiency

At deployment, LoRA merges weights:

$$W_{deploy} = W_0 + BA$$

Inference then proceeds identically to standard Transformer forward pass, adding zero latency—a key distinction from adapters.

Q1.14 Theoretical Insight

If the fine-tuned model lies on a low-dimensional subspace $\mathcal{S}$:

$$\dim(\mathcal{S}) \approx r$$

Then LoRA's optimization implicitly projects updates into that manifold, approximating gradient descent on the "intrinsic subspace" of adaptation.

Q1.15 Comparison with Adapter & Prefix Methods

- Adapters: Add layers → increase depth, latency $\propto O(L)$.
- Prefix-tuning: Reduces effective sequence length (context shrink).
- LoRA: Adds no latency; maintains context length; minimal trainable parameters.

Thus, LoRA optimizes:

$$\text{Efficiency} = \frac{\text{Performance}}{\text{Parameters} + \text{Latency}}$$

maximizing adaptation quality per compute unit.

Q1.16 Empirical Scaling Law

Validation accuracy vs. trainable parameters follows logarithmic scaling:

$$\text{Acc} \propto \log_{10}(|\Theta|)$$

LoRA's curve saturates faster than other methods, showing better efficiency per parameter.

Q1.17 Interpretability of ΔW

The low-rank update ΔW amplifies certain semantic directions latent in W_0 , not random noise. Hence, LoRA can be viewed as *selective feature amplification*, improving generalization.

Q1.18 Theoretical Generalization

If $\Delta W = BA$ and B, A are small, gradient flow remains stable:

$$\frac{dL}{dt} = -\langle \nabla L, \dot{BA} \rangle \approx -\langle \nabla L, B\dot{A} + \dot{B}A \rangle$$

Thus, optimization remains smooth and low-variance, unlike full fine-tuning where gradient variance is higher due to full-dimensional updates.

BERT

Q1.1 Bidirectional Context Modeling

BERT replaces unidirectional LM with *Masked LM*:

$$L_{MLM} = - \sum_{t \in M} \log P_{\theta}(w_t | w_{\setminus t})$$

This allows conditioning on both left and right context.

Unlike left-to-right LM ($P(w_t | w_{<t})$), MLM avoids information leakage via random masking.

Q1.2 Next Sentence Prediction (NSP)

Sentence-level coherence is captured with a binary classification objective:

$$L_{NSP} = -[y \log P_{\theta}(\text{IsNext}) + (1 - y) \log P_{\theta}(\text{NotNext})]$$

Joint training of $L = L_{MLM} + L_{NSP}$ enables contextual alignment across sentence pairs (useful for NLI/QA).

Q1.3 Input Representation Fusion

Each token embedding is the sum of three components:

$$E_i = E_i^{token} + E_i^{segment} + E_i^{position}$$

[CLS] and [SEP] tokens are used for global classification and sentence separation, enabling sequence-pair modeling within a unified Transformer.

Q1.4 Fine-Tuning Objective (Classification Tasks)

For classification tasks (e.g., GLUE):

$$P(y | x) = \text{softmax}(WC)$$
$$L_{task} = - \sum_i \log P(y_i | x_i)$$

where C is the [CLS] token's final hidden state.

All Transformer parameters are fine-tuned jointly.

Q1.5 Span Prediction (Question Answering)

Given question-passage pairs:

$$P_{start}(i) = \frac{e^{S \cdot T_i}}{\sum_j e^{S \cdot T_j}}, P_{end}(i) = \frac{e^{E \cdot T_i}}{\sum_j e^{E \cdot T_j}}$$
$$L_{QA} = -\log P_{start}(i^*) - \log P_{end}(j^*)$$

where T_i is token embedding, S, E are learned vectors, and (i^*, j^*) are ground-truth answer indices.

Q1.6 Efficiency and Model Scaling

Let (L, H, A) denote layers, hidden size, and attention heads.

Empirically, accuracy $\propto$ model capacity:

$$\text{Acc} \propto \log(L \cdot H \cdot A)$$

Despite quadratic attention cost $O(T^2)$, pre-training efficiency is increased via sequence-length scheduling:

90% at 128 tokens, 10% at 512 tokens.

Bart paper

Q1. What is the mathematical objective function used to train BART?

Answer:

BART minimizes the **negative log-likelihood (cross-entropy)** between the predicted output and the original text.

$$\mathcal{L} = - \sum_{t=1}^T \log P_{\theta}(x_t | \tilde{x})$$

Where:

- x_t : the original (true) token at position t
- $\tilde{x}$: the **corrupted input sequence**

- P_θ : model's probability distribution over tokens

□ **Logic:**

This loss measures how well BART reconstructs clean text from noisy input — lower loss = better reconstruction.

Q2. What is the mathematical intuition behind the *denoising autoencoder* used in BART?

Answer:

BART learns a mapping:

$$f_\theta: \tilde{x} \rightarrow x$$

where $\tilde{x}$ is the **noisy text** and x is the **original text**.

The goal is to minimize reconstruction error:

$$\min_{\theta} \mathbb{E}_{\tilde{x} \sim q(\tilde{x}|x)} [-\log P_\theta(x | \tilde{x})]$$

□ **Logic:**

This objective teaches the model to **reverse random corruptions**, making it robust for tasks requiring text understanding and generation.

Q3. Explain mathematically how *attention weights* are computed in BART.

Answer:

Attention measures token relevance:

$$\alpha_i^{(t)} = \frac{e^{h_t^\top h_i}}{\sum_j e^{h_t^\top h_j}}$$

and the **context vector**:

$$c_t = \sum_i \alpha_i^{(t)} h_i$$

Where:

- h_t : hidden state of the current token
- h_i : hidden state of previous token i

□ **Logic:**

Attention assigns **probabilities** to previous words showing which ones are most relevant for predicting the next token.

Q4. How does the BART decoder generate text autoregressively (mathematically)?

Answer:

The decoder predicts the next token using left-to-right conditioning:

$$P(x_t | x_{<t}, \tilde{x}) = \text{softmax}(Wh_t)$$

where h_t is the decoder hidden state at position t .

□ **Logic:**

Each token prediction depends only on **past tokens** (causal masking), allowing BART to **generate text sequentially**.

Q5. What distribution controls span lengths in text infilling?

Answer:

Span lengths l are sampled from a **Poisson distribution**:

$$l \sim \text{Poisson}(\lambda = 3)$$

□ **Logic:**

This stochastic span selection helps the model learn to **predict variable-length gaps**, improving understanding of sentence structure.

Q6. What is *perplexity* and how is it derived from cross-entropy loss?

Answer:

$$PPL = e^{\mathcal{L}/N}$$

where $\mathcal{L}$ is the average cross-entropy loss, and N is the total number of tokens.

□ **Logic:**

- Lower PPL $\rightarrow$ model predictions are more confident and fluent.
 - It measures **how well a model predicts actual text sequences**.
-

Q7. Explain the role of *softmax* in token prediction mathematically.

Answer:

At each position t :

$$P(x_t = v_i | x_{<t}) = \frac{e^{(w_i^T h_t)}}{\sum_j e^{(w_j^T h_t)}}$$

□ **Logic:**

Softmax converts raw scores into probabilities, ensuring the sum over all vocabulary tokens equals 1.

Q8. Write the combined form of encoder-decoder attention in BART.

Answer:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- **Encoder Attention:** uses full (bidirectional) context.
- **Decoder Attention:** restricted (left-to-right).
- **Cross-Attention:** decoder queries encoder outputs.

□ **Logic:**

This allows the decoder to **attend to encoder features** while generating each token, combining understanding with generation.

Q9. Why is *cross-entropy* the ideal choice for BART's pretraining objective?

Answer:

Because it directly measures the **distance between predicted and true probability distributions** over tokens.

Mathematically:

$$H(p, q) = - \sum_x p(x) \log q(x)$$

□ **Logic:**

Minimizing cross-entropy encourages the model to assign high probability to the correct next token, making it ideal for **text reconstruction and sequence generation**.

Q10. How does BART balance between *bidirectional context* and *autoregressive generation*?

Answer:

Through its **hybrid objective**:

$$\text{Encoder: } h = f_{\text{bi}}(\tilde{x}) \text{ and Decoder: } P(x_t \mid x_{<t}, h)$$

□ **Logic:**

The encoder uses **bidirectional attention** for full context understanding, while the decoder uses **causal attention** for sequential text generation — ensuring both comprehension and fluency.

Q11. What mathematical operation allows sentence permutation during noising?

Answer:

If $S = [s_1, s_2, \dots, s_n]$ represents ordered sentences, permutation is:

$$\pi(S) = [s_{\pi(1)}, s_{\pi(2)}, \dots, s_{\pi(n)}]$$

where π is a random permutation function.

□ **Logic:**

This forces the model to **re-learn discourse order and coherence**, improving long-range understanding.

Q12. How does the document rotation transformation work logically and mathematically?

Answer:

If tokens are $[t_1, t_2, \dots, t_n]$, choose a random index k and rotate:

$$[t_k, t_{k+1}, \dots, t_n, t_1, \dots, t_{k-1}]$$

□ **Logic:**

This teaches the model to **recognize true document beginnings**, improving positional understanding.

Q13. Describe the computational complexity of BART's self-attention.

Answer:

Standard attention:

$$O(T^2)$$

where T = sequence length.

With sparse or kernelized attention (like in later improvements):

$$O(T) \text{ or } O(T \log T)$$

□ **Logic:**

Reducing complexity helps scale to longer sequences while retaining contextual sensitivity.

Q14. Why does BART’s decoder include *cross-attention* layers?

Answer:

Mathematically, it allows decoder queries Q_d to attend over encoder keys K_e and values V_e :

$$\text{CrossAttn}(Q_d, K_e, V_e) = \text{softmax}\left(\frac{Q_d K_e^\top}{\sqrt{d_k}}\right) V_e$$

□ **Logic:**

This ensures the generated output depends on the **semantic representation** of the entire input text — crucial for tasks like summarization and translation.

Q15. What mathematical factor differentiates BART from a plain language model?

Answer:

A language model learns:

$$P(x) = \prod_t P(x_t \mid x_{<t})$$

BART learns:

$$P(x \mid \tilde{x}) = \prod_t P(x_t \mid \tilde{x}, x_{<t})$$

□ **Logic:**

Unlike plain LMs, BART’s conditional structure allows it to handle **corrupted input** and perform **conditional text reconstruction**.

✔ **Summary Table — Quick Revision**

Concept	Formula	Logical Role
Cross-Entropy	$(-\sum_t \log P_{\theta}(x_t \mid \tilde{x}))$	
Perplexity	$e^{L/N}$	Measures fluency
Attention Weight	$\frac{e^{h_t^\top h_i}}{\sum_j e^{h_t^\top h_j}}$	Relevance of tokens

Text Infilling	$l \sim \text{Poisson}(\lambda = 3)$	Variable span masking
Softmax	$\frac{e^{z_i}}{\sum_j e^{z_j}}$	Converts logits to probs
Sentence Permutation	$S \rightarrow S_\pi$	Tests coherence understanding
Document Rotation	shift tokens by random index	Tests positional awareness

Q-LORA

Q1. What is the main problem QLORA aims to solve?

Answer:

Full 16-bit finetuning of large models (e.g., 65B parameters) requires **>780 GB GPU memory**, which is infeasible.

QLORA solves this by enabling **4-bit finetuning** without performance loss — reducing memory to **<48 GB** while matching 16-bit accuracy.

□ **Logic:**

It attacks the **computational bottleneck** in LLM finetuning — memory — using **quantization + adapter learning** instead of full weight updates.

Q2. What is the core mathematical principle behind 4-bit quantization?

Answer:

Quantization maps continuous values X_{FP32} to a discrete 4-bit range:

$$X_{Int4} = \text{round}\left(\frac{127}{\text{absmax}(X_{FP32})} X_{FP32}\right)$$

and dequantization reverses it:

$$X_{FP32} = \frac{X_{Int4}}{c_{FP32}}$$

□ **Logic:**

This keeps numerical precision in a smaller bit space, drastically cutting memory while maintaining statistical fidelity.

Q3. Define the concept of “NormalFloat (NF4)” mathematically.

Answer:

NF4 uses **quantile quantization** on normally distributed weights:

$$q_i = \frac{1}{2} [Q_x(\frac{i}{2^k + 1}) + Q_x(\frac{i+1}{2^k + 1})]$$

where Q_x is the quantile function of $N(0,1)$.

□ **Logic:**

Instead of uniform bins, NF4 ensures each quantization level represents **equal probability mass**, optimizing information retention for normally distributed model weights.

Q4. What is Double Quantization (DQ) and how is it computed?

Answer:

DQ quantizes **quantization constants themselves**:

$$c_2^{FP32} \rightarrow c_2^{FP8} \text{ using blocksize 256}$$

Average memory per parameter:

$$\frac{32}{64} \rightarrow \frac{8}{64} + \frac{32}{64 \times 256} = 0.127 \text{ bits/param}$$

□ **Logic:**

This recursive compression reduces memory by ≈ 0.373 bits per parameter — a secondary quantization layer improving storage efficiency.

Q5. Write the main QLORA layer equation combining quantization and LoRA.

Answer:

$$Y_{BF16} = X_{BF16} \cdot \text{doubleDequant}(c_1^{FP32}, c_2^{k\text{-bit}}, W_{NF4}) + X_{BF16} L_1 L_2$$

where:

$$\text{doubleDequant}(c_1, c_2, W) = \text{dequant}(\text{dequant}(c_1, c_2), W)$$

□ **Logic:**

This equation defines forward propagation in QLORA:

- The **base weights** are 4-bit quantized.
- The **LoRA adapters** L_1, L_2 are trainable 16-bit matrices.
It combines **compression + adaptability** mathematically.

Q6. How does QLORA handle backpropagation through quantized weights?

Answer:

Gradients are computed only for adapter weights:

$$\frac{\partial E}{\partial L_i}, \text{ not for } \frac{\partial E}{\partial W}$$

But the partial derivative uses dequantized weights:

$$\frac{\partial X}{\partial W} \text{ is computed in BF16 precision.}$$

□ **Logic:**

Frozen 4-bit weights avoid update cost; gradients flow *through* quantized tensors into adapters — combining efficiency with learning flexibility.

Q7. Explain the LoRA mathematical update used within QLORA.

Answer:

Given a linear layer $Y = XW$:

LoRA adds a low-rank update:

$$Y = XW + sXL_1L_2$$

where $L_1 \in \mathbb{R}^{h \times r}, L_2 \in \mathbb{R}^{r \times o}$.

□ **Logic:**

Only L_1, L_2 are trained $\rightarrow$ low-rank structure approximates full update while freezing the large weight matrix W .

Q8. What is the memory trade-off demonstrated mathematically?

Answer:

Original LLaMA 65B finetuning:

> 780 GB

QLORA (4-bit quantization + adapters):

< 48 GB

Reduction ratio:

Memory saved $\approx 94\%$

□ **Logic:**

Quantization and LoRA jointly compress model memory without degrading performance — mathematically optimizing bits-per-parameter.

Q9. Why is NF4 “information-theoretically optimal”?

Answer:

Because it minimizes quantization error:

$$\text{minimize } \mathbb{E}[(x - \hat{x})^2]$$

under the constraint that bins contain equal probability mass of a normal distribution.

□ **Logic:**

Equal quantile spacing guarantees **maximum information density** per bit — critical for 4-bit precision.

Q10. What mathematical proof shows NF4 outperforms Float4 and Int4?

Answer:

Experimental mean perplexity (PPL) comparison:

Data Type Mean PPL ↓

Int4 34.34

Float4 29.48

NF4 + DQ 27.41

□ **Logic:**

Lower perplexity means better predictive probability distribution — NF4 preserves weight information most efficiently.

Q11. How does QLORA use Elo rating mathematically to evaluate chatbots?

Answer:

Elo update rule:

$$E_A = \frac{1}{1 + 10^{(R_B - R_A)/400}}$$
$$R'_A = R_A + K(S_A - E_A)$$

□ **Logic:**

Each chatbot's rating updates based on win/loss outcomes judged by GPT-4 or humans — quantifying conversational quality mathematically.

Q12. Write the quantization-dequantization transformation chain for a single tensor.

Answer:

$$X_{FP32} \xrightarrow{\text{Quant}} W_{NF4} \xrightarrow{\text{Double Quant}} (c_2^{FP8}, c_1^{FP32}) \xrightarrow{\text{Dequant}} W_{BF16}$$

□ **Logic:**

This chain compresses → stores → expands → computes.

QLORA mathematically cycles between **4-bit storage** and **16-bit computation**.

Q13. What mathematical evidence shows QLORA retains 16-bit accuracy?

Answer:

From experiments:

$$\text{Acc}_{QLORA_NF4} = \text{Acc}_{Full16bit} \pm 0.2\%$$

and

$$PPL_{QLORA_NF4} = 27.41 \text{ vs. } PPL_{Float4} = 29.48$$

□ **Logic:**

The near-zero delta confirms the quantization + adapter combination does not degrade performance statistically.

Q14. Summarize the mathematical efficiency principle of QLORA.

Answer:

$$\text{Performance} \propto \frac{\text{Precision}}{\text{Memory Usage}} \Rightarrow \text{QLORA maximizes this ratio}$$

by

- Reducing bits (quantization),
- Maintaining accuracy (LoRA),
- Managing overflow (Paged Optimizer).

□ **Logic:**

It's a trade-off optimization: **less precision per parameter, more overall parameters per GPU**, achieving a mathematically optimal performance/memory balance.

Q15. What are the main mathematical limitations of QLORA?

Answer:

1. Undefined performance boundary for <4-bit quantization (e.g., 3-bit).
2. Limited benchmark generalization.
3. Evaluation bias in Elo/GPT-based scoring.

□ **Logic:**

Mathematically unresolved — how far quantization can go before accuracy decay becomes irreversible.

✓ **Quick Formula Recap Sheet**

Concept	Equation	Purpose
---------	----------	---------

Quantization	$X_{Int8} = \text{round}(\frac{127}{\text{absmax}(X)}X)$	Reduce bit precision
NF4 Quantile	$q_i = \frac{1}{2}(Q_X(\frac{i}{2^k+1}) + Q_X(\frac{i+1}{2^k+1}))$	Equal probability bins
LoRA Update	$Y = XW + sXL_1L_2$	Low-rank adaptation
Double Quantization	$32/64 \rightarrow 8/64 + 32/(64 \times 256) = 0.127\text{bits}$	Compress constants
Elo Rating	$E_A = \frac{1}{1 + 10^{(R_B - R_A)/400}}$	Model performance measure
NF4 Loss	$\min_{\theta} \mathbb{E}[(x - \hat{x})^2]$	Optimize information